



17-MAVZU · 42 TADAN · II BOB. JAVASCRIPT ASOSLARI

JavaScript'da takrorlanish operatorlari va funksiyalar

for, while, do...while sikllari, break va continue, cheksiz sikl xavfi; funksiya e'lon qilish, parametrlar, return, arrow funksiyalar, ko'rinish doirasi; kalkulyator va son topish o'yini.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- Bir xil ishni ko'p marta bajaruvchi **sikllarni**: `for`, `while`, `do...while`;
- Siklni to'xtatish (`break`) va qadamni o'tkazib yuborish (`continue`);
- **Funksiya** — qayta ishlatiladigan kod bloki yaratishni;
- Parametrlar, `return` va arrow funksiyalarni;
- Kalkulyator va "Sonni top" o'yinini (9 va 10-loyihalar).

Nima uchun sikl kerak?

1 dan 100 gacha sonlarni chiqarish kerak. 100 qator `console.log` yozasizmi? Albatta yo'q! **Sikl** — kodni berilgan shart bajarilguncha **qayta-qayta** bajaruvchi konstruktsiya.



O'xshatish: sport mashg'uloti

Murabbiy: "20 marta o'tirib-tur!" Siz har safar sanaysiz: 1, 2, 3... 20 ga yetganda to'xtaysiz. Bu — `for` sikli: boshlang'ich qiymat (1), shart (20 gacha) va qadam (har safar +1).

for sikli

JavaScript

```
for (boshlanish; shart; qadam) {  
  // takrorlanadigan kod (siklning "tanasi")  
}
```

JavaScript

```
for (let i = 1; i ≤ 5; i++) {  
  console.log(`${i}-marta o'tirib-turdim`);  
}  
console.log("Mashq tugadi! 💪");
```

Qism	Kod	Ma'nosi
Boshlanish	let i = 1	Hisoblagich 1 dan boshlanadi (bir marta bajariladi)
Shart	i ≤ 5	Har qadam oldidan tekshiriladi; false bo'lsa sikl tugaydi
Qadam	i++	Har aylanishdan keyin i bittaga oshadi

Qadamma-qadam: $i=1 \rightarrow 1 \leq 5?$ ha $\rightarrow$ chiqar $\rightarrow i=2 \rightarrow \dots \rightarrow i=5 \rightarrow$ chiqar $\rightarrow i=6 \rightarrow 6 \leq 5?$ yo'q $\rightarrow$ **tugadi**.

Turli sikllar

JavaScript

```
// Teskari sanash
for (let i = 10; i ≥ 1; i--) {
  console.log(i);
}
console.log("🚀 Start!");

// 2 qadam bilan: faqat juft sonlar
for (let i = 2; i ≤ 20; i += 2) {
  console.log(i);
}
```

Yig'indi hisoblash — eng muhim namuna

JavaScript

```
let yigindi = 0; // "hamyon" — avval bo'sh
for (let i = 1; i ≤ 100; i++) {
  yigindi += i; // har bir sonni hamyonga qo'shamiz
}
console.log(`1 dan 100 gacha yig'indi: ${yigindi}`); // 5050
```

Bu **akkumulyator** (yig'uvchi) namunasi: sikldan oldin 0 yoki bo'sh qiymatli o'zgaruvchi, sikl ichida unga qo'shib borish. Ko'p masalalarda kerak bo'ladi.

Ko'paytirish jadvali — sikl ichida sikl

JavaScript

```
for (let i = 1; i ≤ 5; i++) {
  let qator = "";
  for (let j = 1; j ≤ 5; j++) {
    qator += (i * j) + "\t"; // \t — tabulyatsiya (bo'sh joy)
  }
  console.log(qator);
}
```

while va do...while

`while` — **nechta** qadam bo'lishini oldindan bilmaganimizda ishlatiladi: "shart rost ekan, takrorla".

JavaScript

```
// Pul jamg'arish: har oy 50 000 so'm, velosiped 400 000 so'm
let jamgarma = 0;
let oy = 0;
while (jamgarma < 400000) {
  jamgarma += 50000;
  oy++;
}
console.log(`${oy} oyda velosipedga pul yig'iladi 🚲`);
```

JavaScript

```
// do...while — kamida BIR MARTA bajariladi, shart keyin tekshiriladi
let parol;
do {
  parol = prompt("Kamida 6 belgili parol kiriting:");
} while (parol !== null && parol.length < 6);
console.log(parol ? "Parol qabul qilindi ✅" : "Bekor qilindi");
```



Cheksiz sikl!

Agar shart **hech qachon** `false` bo'lmasa, sikl to'xtamaydi va brauzer "qotib" qoladi: `while (true) { }` yoki hisoblagichni oshirishni unutish: `let i = 0; while (i < 5) { console.log(i); } — i++` yo'q! Shunday bo'lsa, brauzer yorlig'ini yoping.

break va continue

JavaScript

```
// break — siklni darhol TO'XTATISH
for (let i = 1; i ≤ 100; i++) {
  if (i % 7 === 0 && i % 5 === 0) {
    console.log(`5 ga ham, 7 ga ham bo'linadigan birinchi son: ${i}`);
    break;
  }
}

// continue — joriy qadamni O'TKAZIB, keyingisiga o'tish
for (let i = 1; i ≤ 10; i++) {
  if (i % 3 === 0) continue; // 3 ga bo'linadiganlarni tashlab ketamiz
  console.log(i);           // 1 2 4 5 7 8 10
}
```

Funksiyalar

Funksiya — nom berilgan, **qayta ishlatiladigan** kod bloki. Bir marta yozasiz — xohlagancha chaqirasiz.



O'xshatish: blender (mikser)

Blenderga meva **solasiz** (parametrlar), tugmani **bosasiz** (chaqirish), ichida maydalanadi (funksiya tanasi), va sizga **sharbat qaytaradi** (return). Har safar blender yasash shart emas — tayyor blenderdan foydalanasiz. Olma solsangiz — olma sharbati, banan solsangiz — banan sharbati.

Funksiya e'lon qilish va chaqirish

JavaScript

```
// 1. E'lon qilish (yaratish) – hali hech narsa bajarilmaydi
function salomlash() {
  console.log("Assalomu alaykum!");
  console.log("Darsga xush kelibsiz!");
}

// 2. Chaqirish – endi bajariladi
salomlash();
salomlash(); // xohlagancha marta
```

Parametrlar — funksiyaga ma'lumot berish

JavaScript

```
function salomlash(ism, sinf) { // ism, sinf – parametrlar
  console.log(`Salom, ${ism}! Siz ${sinf}-sinfdasiz.`);
}

salomlash("Aziza", 8); // "Aziza", 8 – argumentlar
salomlash("Bobur", 9);

// Standart qiymat
function xabar(matn, belgi = " !") {
  console.log(belgi + " " + matn);
}

xabar("Ertaga nazorat ishi"); // ! Ertaga nazorat ishi
xabar("Yangi dars", "📢"); // 📢 Yangi dars
```

return — natija qaytarish

`console.log` faqat ekranga chiqaradi. `return` esa natijani **qaytaradi** — uni o'zgaruvchiga saqlash va boshqa hisoblarda ishlatish mumkin.

JavaScript

```
function kvadrat(son) {  
    return son * son;  
}  
  
const a = kvadrat(5);           // 25  
const b = kvadrat(3) + kvadrat(4); // 9 + 16 = 25  
console.log(a, b);  
  
function bahoAniqla(ball) {  
    if (ball ≥ 86) return 5;    // return funksiyani darhol TUGATADI  
    if (ball ≥ 71) return 4;  
    if (ball ≥ 56) return 3;  
    return 2;  
}  
console.log(bahoAniqla(92), bahoAniqla(60), bahoAniqla(30));
```



`return` dan keyingi kod bajarilmaydi. `return` siz funksiya `undefined` qaytaradi.

Arrow (strelkali) funksiyalar — qisqa yozuv

JavaScript

```
// Oddiy funksiya  
function qoshish(a, b) {  
    return a + b;  
}  
  
// Arrow funksiya — xuddi shu narsa  
const qoshish2 = (a, b) ⇒ {  
    return a + b;  
};  
  
// Bitta ifoda bo'lsa — eng qisqa (return avtomatik)  
const qoshish3 = (a, b) ⇒ a + b;  
const ikkilantir = son ⇒ son * 2;    // bitta parametr — qavs ixtiyoriy  
  
console.log(qoshish(2, 3), qoshish2(2, 3), qoshish3(2, 3), ikkilantir(21));
```

Zamonaviy kodda arrow funksiyalar juda ko'p uchraydi, ayniqsa 18-mavzudagi massiv metodlari bilan.

Ko'rinish doirasi (scope)

JavaScript

```
const maktab = "5-maktab";           // global — hamma joyda ko'rinadi

function tanishtir() {
  const ism = "Laylo";                // lokal — faqat funksiya ichida
  console.log(`${ism}, ${maktab}`); // ✅ ikkalasi ham ko'rinadi
}

tanishtir();
// console.log(ism);                // ❌ ReferenceError: ism is not defined
```

Funksiya ichida yaratilgan o'zgaruvchi — uning "shaxsiy xonasi". Tashqaridan unga kirib bo'lmaydi. Bu yaxshi: turli funksiyalar bir xil nomli o'zgaruvchilar bilan bir-biriga xalaqit bermaydi.

Yaxshi funksiya qoidalari

- **Bitta funksiya — bitta vazifa.** `yuzaHisobla()` yuzani hisoblasin, chiqarmasin.
- **Nomi fe'l bilan:** `hisobla`, `tekshir`, `korsat`, `getBaho`.
- **Takrorlanadigan kodni funksiyaga chiqaring** (DRY — Don't Repeat Yourself).

Amaliy mashg'ulot

1. Kalkulyator (9-loyiha, konsol versiyasi)

JavaScript

```
function hisobla(a, amal, b) {
  switch (amal) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/":
      if (b === 0) return "Nolga bo'lib bo'lmaydi!";
      return a / b;
    case "%": return a % b;
    case "^": return a ** b;
    default: return "Noma'lum amal";
  }
}

let davom = true;
while (davom) {
  const a = Number(prompt("Birinchi son:"));
  const amal = prompt("Amal (+ - * / % ^):");
  const b = Number(prompt("Ikkinchi son:"));

  if (Number.isNaN(a) || Number.isNaN(b)) {
    alert("Iltimos, son kiriting!");
  } else {
    alert(`${a} ${amal} ${b} = ${hisobla(a, amal, b)}`);
  }
  davom = confirm("Yana hisoblaymizmi?");
}
```

2. "Sonni top" o'yini (10-loyiha)

JavaScript

```
const yashirinSon = Math.floor(Math.random() * 100) + 1; // 1..100
const MAKS_URINISH = 7;
let urinish = 0;
let topildi = false;

alert("Men 1 dan 100 gacha son o'yladim. Topa olasizmi? Sizda 7 ta urinish bor!");

while (urinish < MAKS_URINISH && !topildi) {
  const javob = prompt(`${urinish + 1}-urinish. Sizning taxminingiz:`);
  if (javob === null) break; // "Bekor qilish" bosildi
  const son = Number(javob);
  urinish++;

  if (son === yashirinSon) {
    topildi = true;
    alert(`🎉 Topdingiz! Bu ${yashirinSon} edi. Urinishlar: ${urinish}`);
  } else if (son < yashirinSon) {
    alert("⬆ Kattaroq son o'ylang");
  } else {
    alert("⬇ Kichikroq son o'ylang");
  }
}

if (!topildi) alert(`😞 Urinishlar tugadi. Javob: ${yashirinSon}`);
```

Savol: nima uchun 7 ta urinish 1–100 oralig'i uchun doim yetarli? (Maslahat: har safar o'rtadagi sonni ayting — oralig ikki baravar qisqaradi. Bu **ikkilik qidiruv** algoritmi.)

3. Mustaqil topshiriqlar

- 1 `faktorial(n)` funksiyasini yozing: $5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$.
- 2 `tubmi(n)` — son tub ekanini tekshiruvchi funksiya (faqat 1 ga va o'ziga bo'linadi). Uning yordamida 1–100 oralig'idagi barcha tub sonlarni chiqaring.
- 3 **FizzBuzz** (dasturchilar suhbatida mashhur savol): 1 dan 30 gacha sonlarni chiqaring, lekin 3 ga bo'linsa "Fizz", 5 ga — "Buzz", ikkalasiga — "FizzBuzz".

Ko'p uchraydigan xatolar

- ❌ Funksiyani e'lon qilib, chaqirishni unutish (yoki `salomlash` deb, qavssiz yozish).
- ❌ `return` o'rniga `console.log` — natijani boshqa hisoblarda ishlatib bo'lmaydi (`undefined`).
- ❌ Sikl shartida xato: `for (let i = 0; i ≤ 10; i--)` — i kamayib boradi, shart doim rost → cheksiz sikl.
- ❌ `for (let i = 1; i < 10; i++)` 10 ni ham chiqaradi deb o'ylash — `<` va `≤` farqiga e'tibor bering.

O'zingizni tekshiring

? `for (let i = 0; i < 5; i++)` sikli necha marta bajariladi?

- A. 4
- B. 6
- C. 5
- D. Cheksiz

? Siklni darhol to'xtatuvchi kalit so'z?

- A. `break`
- B. `continue`
- C. `stop`
- D. `return`

? Kamida bir marta bajarilishi kafolatlangan sikl qaysi?

- A. `for`
- B. `while`
- C. `if`
- D. `do...while`

? `const f = x => x * 3; console.log(f(4));` natijasi?

- A. 7
- B. 12
- C. 43
- D. undefined

? Funksiya natijasini chaqirgan joyga qaytaruvchi kalit so'z?

- A. `console.log`
- B. `break`
- C. `return`
- D. `output`

? Quyidagi kod nima chiqaradi? `let s = 0; for (let i = 1; i ≤ 4; i++) s += i; console.log(s);`

- A. 4
- B. 10
- C. 24
- D. 1234

Uyga vazifa

1. Kalkulyatorga "tarix" qo'shing: barcha hisob-kitoblar bir satrga yig'ilib, oxirida ko'rsatilsin.
2. "Sonni top" o'yiniga qiyinlik darajasini qo'shing: oson (1–20, 5 urinish), o'rta (1–100, 7), qiyin (1–1000, 10).
3. `raqamlarYigindisi(n)` funksiyasini yozing: $1234 \rightarrow 1+2+3+4 = 10$. (Maslahat: `% 10` va `Math.floor(n / 10)`.)

Yodda tuting

- `for (let i = 1; i ≤ n; i++)` — necha marta takrorlash ma'lum bo'lsa; `while (shart)` — noma'lum bo'lsa; `do...while` — kamida bir marta.
- `break` — to'xtatish, `continue` — keyingi qadamga o'tish. Cheksiz sikldan ehtiyot bo'ling.
- Funksiya: `function nom(param) { return natija; }` yoki `const nom = (param) ⇒ natija;`
- `return` natija qaytaradi va funktsiyani tugatadi. Funksiya ichidagi o'zgaruvchilar — lokal.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. `for (let i = 0; i < 5; i++)` sikli necha marta bajariladi? — **C) 5**
`i = 0, 1, 2, 3, 4` → 5 marta.
2. Siklni darhol to'xtatuvchi kalit so'z? — **A) break**
3. Kamida bir marta bajarilishi kafolatlangan sikl qaysi? — **D) do...while**
4. `const f = x ⇒ x * 3; console.log(f(4));` natijasi? — **B) 12**
5. Funksiya natijasini chaqirgan joyga qaytaruvchi kalit so'z? — **C) return**
6. Quyidagi kod nima chiqaradi? `let s = 0; for (let i = 1; i <= 4; i++) s += i; console.log(s);` — **B) 10**
`1 + 2 + 3 + 4 = 10.`